SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	NARRAVULA NAVYA SRI	**Reg. No.:**	192324220
Section / Batch:	AI & DS	**Date:**	23-07-26

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
	GRAND TOTAL:		**100 Marks**

SECTION A – SCENARIO BASED ASSESSMENT

Q1. A financial system maintains sorted transaction data for quick lookup using Binary Search, but frequent updates disrupt the order. a) Explain how Binary Search operates on sorted data. b) Analyze its performance when order is maintained versus disturbed. c) Discuss the importance of maintaining sorted data.

Q2. A plagiarism detection tool compares patterns across multiple large documents using brute-force string matching. a) Explain how the matching process works. b) Analyze the time complexity. c) Discuss inefficiency in large-scale comparisons.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46
					Overall Total (100)	**86**

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

A financial system maintains sorted transaction data for quick lookup using Binary Search, but frequent updates disrupt the order. a) Explain how Binary Search operates on sorted data. b) Analyze its performance when order is maintained versus disturbed. c) Discuss the importance of maintaining sorted data.

A financial system maintains sorted transaction data for quick lookup using Binary Search, but frequent updates disrupt the order.

a) Explain how Binary Search operates on sorted data.

Definition

Binary Search is an efficient searching algorithm used to find an element in a sorted array or list. Instead of checking every element one by one, it repeatedly divides the search space into two equal halves until the target element is found or determined to be absent.

Working Process

Assume the sorted transaction IDs are:

[102, 115, 128, 143, 156, 172, 185, 201]

Suppose we want to search for 156.

Step 1:

- Low = 0
- High = 7
- Middle = (0 + 7) // 2 = 3
- Middle value = 143

Since 156 > 143, search the right half.

Step 2:

- Low = 4
- High = 7
- Middle = (4 + 7) // 2 = 5
- Middle value = 172

Since 156 < 172, search the left half.

Step 3:

- Low = 4
- High = 4
- Middle = 4
- Middle value = 156

Target found.

Algorithm

1. Set two pointers:
    - Low = first element
    - High = last element
2. Find the middle element.
3. Compare:
    - If equal → element found.
    - If the target is smaller → search left half.
    - If the target is larger → search right half.
4. Repeat until the element is found or Low > High.

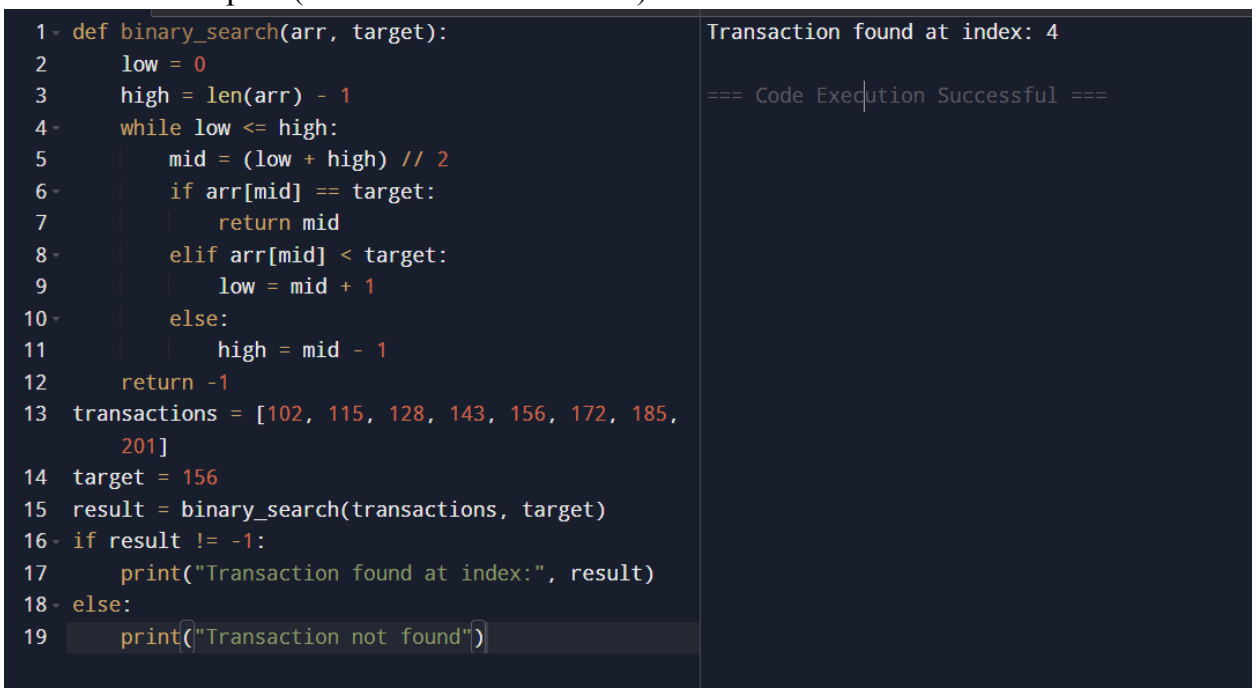
Binary Search Algorithm (Pseudo Code)

```
BinarySearch(arr, target)
low = 0
high = length(arr)-1
while low <= high
   mid = (low + high) // 2
   if arr[mid] == target
      return mid
   else if arr[mid] < target
      low = mid + 1
   else
      high = mid - 1
return -1
```

Python Code and Implementation:

```
def binary_search(arr, target):
   low = 0
   high = len(arr) - 1
   while low <= high:
      mid = (low + high) // 2
```

```
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
transactions = [102, 115, 128, 143, 156, 172, 185, 201]
target = 156
result = binary_search(transactions, target)
if result != -1:
    print("Transaction found at index:", result)
else:
    print("Transaction not found")
```



b) Analyze its performance when order is maintained versus disturbed.

Case 1: Sorted Data (Order Maintained)

- Binary Search works efficiently because the data is sorted.
- Each comparison removes half of the remaining elements.

Example:

For 1,000,000 transactions:

- First comparison → 500,000 remain
- Second → 250,000
- Third → 125,000

Only about 20 comparisons are needed.

Time Complexity

Case	Complexity
Best Case	**O(1)**
Average Case	**O(log n)**
Worst Case	**O(log n)**

This makes Binary Search one of the fastest searching algorithms for sorted data.

Case 2: Unsorted Data (Order Disturbed)

If frequent updates insert transactions without maintaining order:

Example:

[102, 143, 115, 156, 128]

The list is no longer sorted.

Binary Search assumes:

- The left half contains smaller values.
- The right half contains larger values.

Since this assumption is false, Binary Search may:

- Return incorrect results.
- Fail to find an existing transaction.

Therefore, Binary Search cannot be applied reliably on unsorted data.

To search unsorted data, Linear Search is required.

Linear Search Complexity

Case	Complexity
Best Case	O(1)
Average Case	O(n)
Worst Case	O(n)

Comparison

Feature	Sorted Data	Unsorted Data
Binary Search Applicable	Yes	No
Search Speed	Very Fast	Cannot be used correctly
Complexity	O(log n)	O(n) using Linear Search
Reliability	High	Low

c) Discuss the importance of maintaining sorted data.

Maintaining sorted data is essential because many efficient algorithms depend on the sorted order.

1. Faster Searching

Sorted data enables Binary Search, reducing search time from O(n) to O(log n).

Example:

Searching among 1 million transactions:

- Linear Search → up to 1,000,000 comparisons.
- Binary Search → about 20 comparisons.

2. Better Performance

Financial systems process thousands of transactions every second.

Sorted data allows:

- Faster transaction lookup.
- Quicker account verification.
- Efficient report generation.

3. Efficient Data Processing

Many operations become easier:

- Finding minimum and maximum values.
- Range queries.
- Duplicate detection.
- Merging datasets.

4. Supports Other Algorithms

Several algorithms require sorted input, including:

- Binary Search
- Merge Search techniques
- Two-pointer methods
- Efficient range searching

5. Improved Scalability

As transaction records increase into millions, maintaining sorted order keeps lookup performance high and prevents system slowdowns.

Challenges

Frequent updates (insertions and deletions) may disturb the sorted order.

Possible solutions include:

- Re-sort the data periodically.
- Insert new records in the correct position.
- Use balanced trees or indexed databases that automatically maintain sorted order.

Conclusion

Maintaining sorted data is crucial because Binary Search depends entirely on sorted order. When the data remains sorted, searches are extremely fast (O(log n)). If the order is disturbed, Binary Search becomes unreliable, and the system must resort to slower algorithms such as Linear Search (O(n)), reducing overall efficiency.

A plagiarism detection tool compares patterns across multiple large documents using brute-force string matching. a) Explain how the matching process works. b) Analyze the time complexity. c) Discuss inefficiency in large-scale comparisons.

A plagiarism detection tool compares patterns across multiple large documents using Brute-Force String Matching.

a) Explain how the matching process works.

Definition

Brute-Force String Matching (also called the Naive String Matching algorithm) is the simplest pattern matching technique.

It checks every possible position in the text where the pattern could occur.

No preprocessing or indexing is performed.

Working Process

Suppose:

Text:

THIS IS A PLAGIARISM DETECTION SYSTEM

Pattern:

PLAGIARISM

The algorithm compares the pattern starting from the first character.

If a mismatch occurs, it shifts the pattern by one position and repeats.

The process continues until:

- Pattern is found, or
- The entire text has been checked.

Step-by-Step

Example:

Text : ABCABCDABC

Pattern : ABCD

Alignment 1:

```
ABCABCDABC
ABCD
```

Mismatch.

Shift one position.

Alignment 2:

```
ABCABCDABC
 ABCD
```

Mismatch.

Continue.

Alignment 4:

```
ABCABCDABC
  ABCD
```

Complete match found.

Algorithm

1. Start from the first character.
2. Compare each pattern character with the text.
3. If all characters match:
    - Pattern found.
4. If mismatch occurs:
    - Shift the pattern by one position.
5. Repeat until the end of the text.

Python Implementation

```
def brute_force_search(text, pattern):
    n = len(text)
    m = len(pattern)
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
```

```
            j += 1
        if j == m:
            return i
    return -1
text = "THIS IS A PLAGIARISM DETECTION SYSTEM"
pattern = "PLAGIARISM"
position = brute_force_search(text, pattern)
if position != -1:
    print("Pattern found at index:", position)
else:
    print("Pattern not found")
```

```
def brute_force_search(text, pattern):
    n = len(text)
    m = len(pattern)
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            return i
    return -1
text = "THIS IS A PLAGIARISM DETECTION SYSTEM"
pattern = "PLAGIARISM"
position = brute_force_search(text, pattern)
if position != -1:
    print("Pattern found at index:", position)
else:
    print("Pattern not found")
```

```
Pattern found at index: 10

=== Code Execution Successful ===
```

b) Analyze the time complexity.

Let:

- **n** = length of the text
- **m** = length of the pattern

The algorithm checks every possible alignment.

Maximum alignments:

$n - m + 1$

Each alignment may compare all **m** characters.

Therefore,

Worst Case

$O((n - m + 1) \times m)$

Approximately:

$O(nm)$

Best Case

If the pattern matches immediately:

$O(m)$

or considered **O(1)** with respect to the number of alignments.

Average Case

Approximately:

$O(nm)$

Complexity Table

Case	Time Complexity
Best	O(m)
Average	O(nm)
Worst	O(nm)

Space Complexity

Only a few variables are used.

$O(1)$

(Constant memory)

c) Discuss inefficiency in large-scale comparisons.

Brute-force string matching becomes inefficient when dealing with multiple large documents.

1. Large Number of Comparisons

For every possible position in the text, the entire pattern may be compared.

Example:

- Text = 5,000,000 characters
- Pattern = 500 characters

Possible alignments:

$$4{,}999{,}501$$

Each alignment may require up to 500 character comparisons, resulting in billions of comparisons in the worst case.

2. Multiple Documents

Suppose:

- 10,000 documents
- Each compared with every other document

Number of comparisons:

$$10{,}000 \times 9{,}999 / 2 \approx 49{,}995{,}000$$

Each document pair requires many string comparisons, making execution extremely slow.

3. Repeated Comparisons

After a mismatch, the algorithm shifts by only one character and starts comparing again from the beginning of the pattern.

Previously matched characters are not reused, causing unnecessary repeated work.

4. Poor Scalability

As the number and size of documents increase, the execution time grows rapidly.

This makes brute-force methods unsuitable for:

- University plagiarism detection.
- Large digital libraries.
- Search engines.
- Online code plagiarism systems.

5. High Processing Time

Comparing long documents may take significant time, especially when millions of characters must be examined across many document pairs.

Better Alternatives

Modern plagiarism detection systems often use more efficient algorithms such as:

- **Knuth–Morris–Pratt (KMP):** Avoids rechecking previously matched characters; **O(n + m)**.
- **Rabin–Karp:** Uses hashing to compare substrings efficiently; average **O(n + m)**, with worst-case **O(nm)** if many hash collisions occur.
- **Boyer–Moore:** Skips sections of the text using heuristic rules, often performing much faster than brute force in practice.
- **Suffix Arrays/Trees:** Useful for large-scale text indexing and repeated searches.
- **Fingerprinting and n-gram techniques:** Commonly used in plagiarism detection to compare document similarity efficiently.

Conclusion

Brute-force string matching is simple and easy to implement but is computationally expensive. Its worst-case time complexity of **O(nm)** makes it inefficient for large documents and large document collections because it repeatedly compares characters from the beginning after each mismatch. Consequently, modern plagiarism detection systems typically employ more advanced algorithms and indexing techniques to improve speed and scalability.